PROTOFIRE

October 2026

Smart contract audit

rain.deploy

Content

Project description **3**
Executive summary **3**
Reviews **3**
Technical analysis and findings **5**
Security findings **6**
**** Critical 6
*** High 6
** Medium 6
* Low 6
L01 - Fresh forks prevent verification of newly simulated deployments 6
General Risks and Assumptions **8**
Approach and methodology **9**

Project description

The rain.deploy repository provides a Solidity library and Foundry tooling for deterministic deployment of smart contracts across multiple EVM-compatible networks, such as Arbitrum, Base, Ethereum, Flare, HyperEVM and Polygon. It uses the Zoltu deterministic deployment proxy, so each contract lands at the same address on every supported chain. The repository also deploys two on-chain contracts of its own: AddressRegistry, a root-controlled registry that maps names to addresses, and MigrationRegistry, a permissionless record of applied migrations, kept per writer and per namespace.

The tooling generates and freezes versioned snapshots of each release. Before deploying, it checks the snapshot against the compiled source, the network dependencies and the bytecode hashes, then deploys idempotently to each network. Test suites verify that every release is live on every supported network, so production deployments match local compilations byte-for-byte. Helper scripts and manually triggered CI workflows cover broadcasting, explorer verification and HyperEVM/HyperCore setup.

Executive summary

Type	Library
Languages	Solidity
Methods	Architecture Review, Manual Review, Unit Testing, Functional Testing, Automated Review
Documentation	README.md
Repositories	https://github.com/rainlanguage/rain.deploy

Reviews

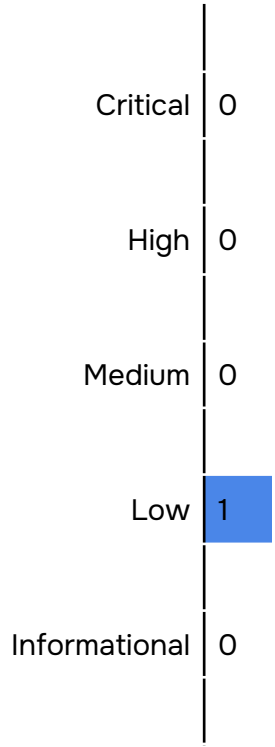
Date	Repository	Commit
09/02/26	rain.deploy	1af8ca2a981c2f67844f343e224f4c7b1969de1c
06/03/26	rain.deploy	7b32ed0a79467ca0406b31e308deec070713b0fe
27/05/26	rain.deploy	506f2a3f0963da8f8c1323e5e584d9161bbb7a95 **tag v0.1.3**
05/10/26	rain.deploy	70e442636e052c9ee93e243862ccbd3c974b8716 **tag sol-v0.1.11**

16/10/26	rain.deploy	18c492a7415a18842c27799c9848b729ae8b970c **tag sol-v0.1.15**

Scope

Contracts
src/abstract/BuildScript.sol src/abstract/RainDeployBroadcast.sol src/abstract/RainDeploySuitesBase.sol src/abstract/RainDeployVerify.sol src/abstract/RainDeployVerifyBase.sol src/abstract/RainDeployVerifyChain.sol src/abstract/RainDeployVerifySnapshot.sol src/abstract/RainDeployVerifySnapshotBase.sol src/abstract/RegistryDeploySuites.sol src/concrete/AddressRegistry.sol src/concrete/MigrationRegistry.sol src/lib/LibAddressRegistry.sol src/lib/LibAddressRegistryDeploy.sol src/lib/LibAddressRegistryReleased.sol src/lib/LibHyperCore.sol src/lib/LibMigrationRegistry.sol src/lib/LibMigrationRegistryDeploy.sol src/lib/LibMigrationRegistryReleased.sol src/lib/LibRainDeploy.sol src/lib/LibRainDeploySnapshot.sol src/lib/LibReleasedSuites.sol src/interface/IAddressRegistryV1.sol src/interface/IMigrationRegistryV1.sol src/interface/IMigrationRegistryV2.sol

Technical analysis and findings



Security findings

**** Critical

No critical severity issue found.

*** High

No high severity issue found.

** Medium

No medium severity issue found.

* Low

L01 - Fresh forks prevent verification of newly simulated deployments

The function `checkResolvedAddressesOnNetworks()` creates new forks instead of reusing the forks on which `deployAndBroadcast()` simulated the deployment. The NatSpec recommends running verification after `deployAndBroadcast()`, but does not distinguish same-script simulation checks from verification after deployment transactions are confirmed. Calling the helpers sequentially within the same script fails for a first-time deployment.

For a consumer calling both helpers within the same Solidity script, the execution sequence is:

1. `deployAndBroadcast()` creates a fork for each target network.
2. It simulates the deployment on those forks and records transactions for subsequent broadcasting.
3. The function returns, but the real deployment transactions have not yet been submitted. `vm.stopBroadcast()` stops transaction recording; it does not wait for a mined deployment.
4. `checkResolvedAddressesOnNetworks()` creates fresh forks from the RPC endpoints.
5. For a first-time deployment, the contract is absent from those fresh forks because it does not exist on the real network yet.
6. The address getter call returns empty data. The helper expects a 32-byte address response and reverts with `ResolvedAddressReadFailed`.
7. Script execution fails, preventing Foundry from broadcasting the collected deployment transactions.

The existing verification test calls `vm.makePersistent(address(consumer))`, making the target available across fork switches. This does not represent the deployment helper's behavior and consequently hides the integration problem.

Impact:
Consumers calling both helpers sequentially within the same Solidity script can encounter blocked deployments even when the deployment and expected configuration are correct.

This issue does not, by itself, expose funds, bypass authorization, or corrupt on-chain state. The failure occurs during simulation, before broadcasting. Its consequences are failed automation, deployment delays, and troubleshooting effort.

The repository's current `RainDeployBroadcast.run()` does not invoke this verification helper, so the issue does not affect every normal deployment.

Path: src/lib/LibRainDeploy.sol, test/src/lib/LibRainDeploy.t.sol

Recommendation:
Define and document two separate verification stages:

- Simulation verification: perform checks within each deployment fork, either directly in the deployment loop or by retaining and returning its fork ID.
- Confirmed deployment verification: execute a separate script after deployment transactions are confirmed, using fresh forks to inspect actual network state.

Add an integration test that calls deployment followed by verification without making the target persistent. Preserve independent state for each network; making the deployed target persistent could conceal differences between networks.

Found in: 70e44263

Status: Fixed in 18c492a7

General Risks and Assumptions

- Deterministic Deployment Limitation: The Zoltu factory uses CREATE2 with a fixed zero salt. Identical creation code, including encoded constructor arguments, therefore maps to one address per network. This tooling does not support selecting a different salt to deploy additional instances. Additional instances require different creation code, for example through different constructor arguments, or another deployment mechanism.

- Deployment Race / Front-running: The factory is permissionless, so another party can deploy the same creation code before the intended deployment transaction is included. The library skips deployment when code already exists during simulation and checks its runtime code hash against the expected value. However, a deployment occurring after that check can cause the pending deployment transaction to revert due to a CREATE2 collision, resulting in failed automation and potentially wasted transaction fees. Constructor msg.sender is the factory address regardless of who submits the transaction; front-running does not change it to the front-runner's address.

- Factory Availability and Integrity: New deployments require the Zoltu factory at 0x7A0D94F55792C434d74a40883C6ed8545E406D12 with the runtime code hash expected by the library. Missing or mismatched factory code causes deployment to revert. Networks without the canonical factory must have it deployed before this tooling can perform new deployments. The factory check is skipped when the target contract already exists, but the target's runtime code hash is still verified. A normal network hard fork does not inherently remove an existing factory.

Approach and methodology

To establish a uniform evaluation, we define the following terminology in accordance with the OWASP Risk Rating
Methodology:

	**Likelihood** Indicates the probability of a specific vulnerability being discovered and exploited in real-world scenarios
	**Impact** Measures the technical loss and business repercussions resulting from a successful attack
	**Severity** Reflects the comprehensive magnitude of the risk, combining both the probability of occurrence (likelihood) and the extent of potential consequences (impact)

Likelihood and impact are divided into three levels: High H, Medium M, and Low L. The severity of a risk is a blend of these two factors, leading to its classification into one of four tiers: Critical, High, Medium, or Low.

When we identify an issue, our approach may include deploying contracts on our private testnet for validation through testing. Where necessary, we might also create a Proof of Concept PoC to demonstrate potential exploitability. In particular, we perform the audit according to the following procedure:

	**Advanced DeFi Scrutiny** We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs
	**Semantic Consistency Checks** We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
	**Security Analysis** The process begins with a comprehensive examination of the system to gain a deep understanding of its internal mechanisms, identifying any irregularities and potential weak spots.